

Rapport du projet SAE 3.01

Moustapha Mbaye et Assane Ndiaye

Introduction

Dans le cadre de cette **SAE (Situation d'Apprentissage et d'Évaluation)**, nous avons travaillé sur l'intégration et la configuration d'un dispositif **LoRaWAN** en utilisant la plateforme **The Things Network**. L'objectif principal de cette étude était de mettre en place une application permettant la transmission et l'analyse de données via un réseau **LoRaWAN**, un protocole de communication longue portée et basse consommation, couramment utilisé pour l'**Internet des Objets (IoT)**.

Dans ce rapport, nous détaillerons les différentes étapes du projet, depuis la création de l'application et du dispositif, jusqu'à l'installation des bibliothèques et l'analyse des résultats obtenus. Nous avons également mis en place des **webhooks** pour l'automatisation du transfert de données et un tableau de bord permettant de visualiser ces informations en temps réel.

Création application et device

Nous avons commencé par créer une application et un device pour notre projet. Cette étape consiste à configurer les paramètres de connexion et d'identification du device dans la plateforme utilisée.

- L'application "assmou" est créée avec un identifiant unique.
- Elle servira à gérer les dispositifs LoRaWAN connectés.


THE THINGS STACK
SANDBOX

Home

Applications

Gateways

Ctrl

K

Top applications

+

 LorawanSaubinTuzi

 Destinee Filank

 andredulac1

 Airbarb

 dumasmillot

 Voltmètre Distant SAE

[Show more](#)


Resources

eu1 • v3.33.0.3c8e962fdd

Applications > Create application

Owner*

rt2-iut-roanne-2024

Application ID*

assmou-sae301

Application name

assmou

Description

Optional application description; can also be used to save notes about the ap

Create application

Paramétrage du device sur The Things Stack

- Choix du plan de fréquence "Europe 863-870 MHz" pour une compatibilité en Europe.
- Sélection de la version LoRaWAN 1.0.3 et des paramètres régionaux RP001.

Home Applications Gateways

Ctrl K

← assmou

Application overview

End devices

Live data

Payload formatters

Integrations

API keys

General settings

☐ Select the end device in the LoRaWAN Device Repository
 ☒ Enter end device specifics manually

Frequency plan ⓘ *
 Europe 863-870 MHz (SF9 for RX2 - recommended)

LoRaWAN version ⓘ *
 LoRaWAN Specification 1.0.3

Regional Parameters version ⓘ *
 RP001 Regional Parameters 1.0.3 revision A

[Show advanced activation, LoRaWAN class and cluster settings](#)

Provisioning information

JoinEUI ⓘ *
 70 B3 D5 7E D0 00 00 1F Confirm

Configuration des identifiants du device.

- Le **JoinEUI** est défini pour identifier le réseau.
- Le **DevEUI** est généré automatiquement pour identifier le dispositif.
- L'**AppKey** est utilisée pour sécuriser les communications entre le device et le réseau.
- L'identifiant du device "mouass-sae301" est enregistré.

Home Applications Gateways

Ctrl K

← assmou

Application overview

End devices

Live data

Payload formatters

Integrations

API keys

General settings

Provisioning information

JoinEUI ⓘ *
 70 B3 D5 7E D0 00 00 1F Reset
 This end device can be registered on the network

DevEUI ⓘ *
 70 B3 D5 7E D0 06 D5 E0 Generate 1/50 used

AppKey ⓘ *
 73 60 FB B0 48 0E BC C1 EB 24 BC A9 D4 89 8C 90 Generate

End device ID ⓘ *
 mouass-sae301

After registration

☒ View registered end device
 ☐ Register another end device of this type

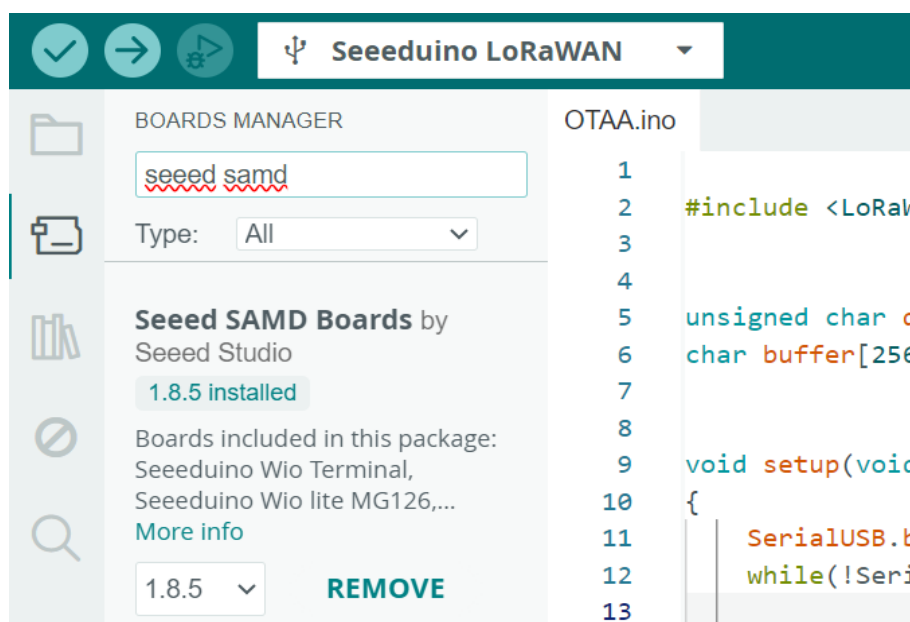
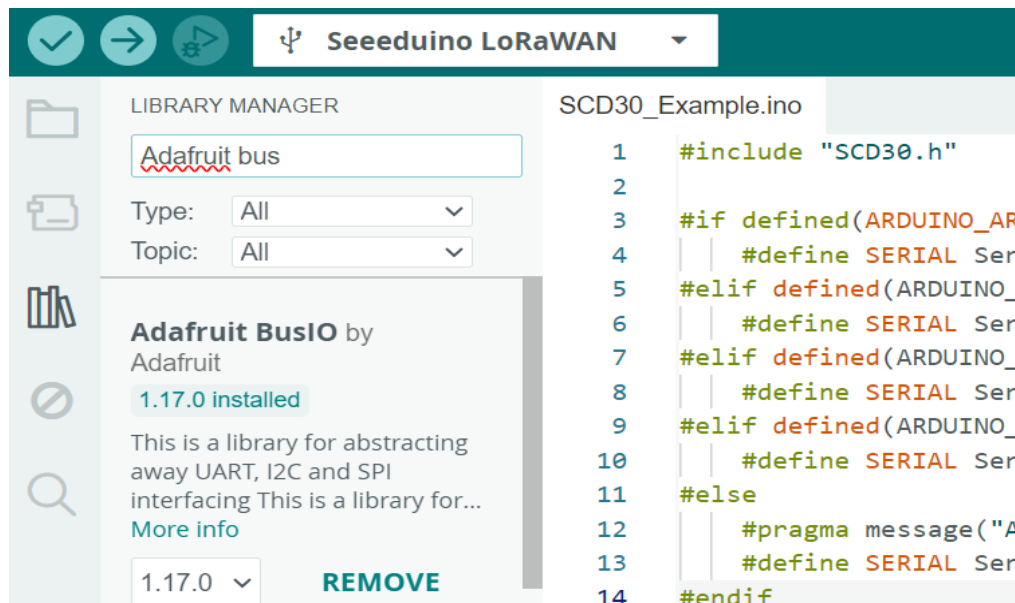
Register end device

Resources

eu1 • v3.33.0.3c8e962fdd

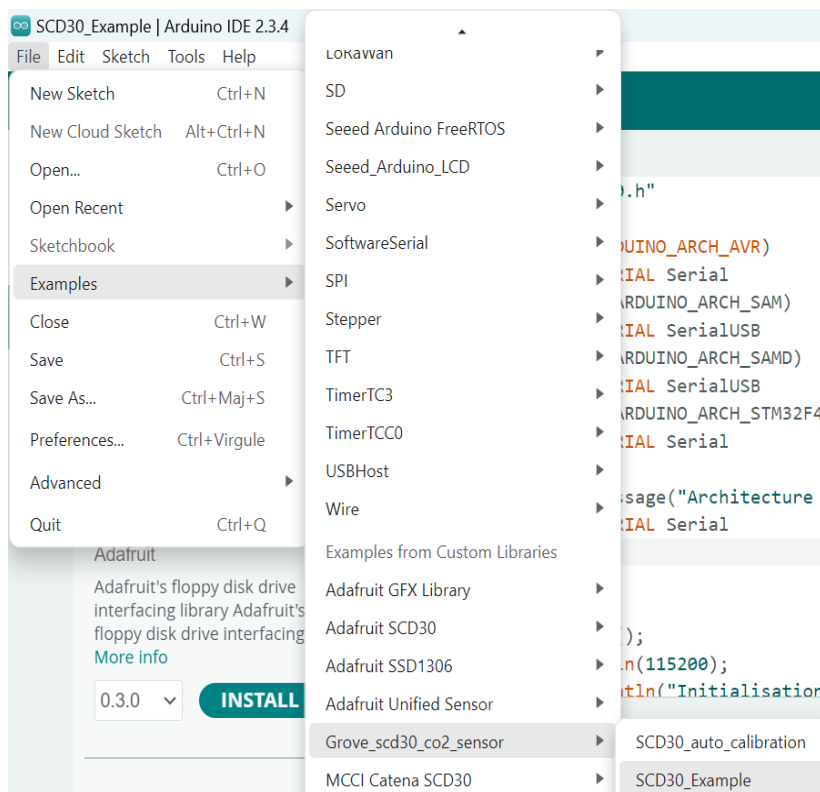
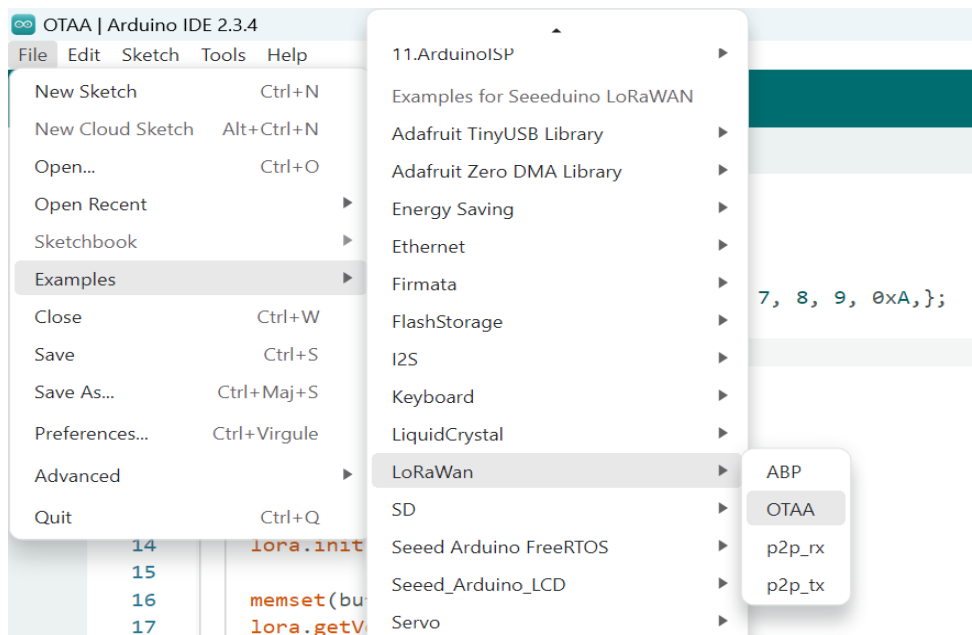
Installation des bibliothèques et du Board

Pour assurer la compatibilité et le bon fonctionnement de notre programme, nous avons installé les bibliothèques et board nécessaires via un gestionnaire de paquets.



Programmes d'exemples OTAA et SCD30

Nous avons testé des programmes d'exemple utilisant les protocoles OTAA et le capteur SCD30 pour vérifier le bon fonctionnement de notre système.



Tests des programmes OTAA et SCD30

Exécution des programmes de test.

- Vérification de la transmission des données via OTAA.
- Test du capteur SCD30 pour la mesure de la qualité de l'air.

Programme final

Après les tests initiaux, nous avons écrit le programme final intégrant toutes les fonctionnalités requises.

Dans un premier temps, on aura une partie du code dédiée à la configuration des identifiants pour la communication LoRaWAN et l'importation des bibliothèques nécessaires au fonctionnement.

Code_sae301.ino

```
1  #include <LoRaWAN.h>
2  #include <Adafruit_SCD30.h>
3  #include <Wire.h>
4
5  // Identifiants LoRaWAN
6  const char *AppEUI = "70B3D57ED000001F";
7  const char *DevEUI = "70B3D57ED006D5E0";
8  const char *AppKey = "7360FBB0480EBCC1EB24BCA9D4898C90";
```

Ainsi, on pourra programmer la Fonction setup() qui comprend :

- **Une partie d'initialisation de la communication** où le port série sera initialisé pour afficher des messages de débogage. Le capteur SCD30 est également configuré et en cas d'échec, le programme est bloqué.
- **Une partie de configuration de LoRaWAN** où le mode OTAA est configuré pour permettre une activation sécurisée et une boucle vérifie la connexion au réseau LoRaWAN. Après 10 échecs, la carte redémarre automatiquement.

Code_sae301.ino

```
10 // Objet pour le capteur SCD30
11 Adafruit_SCD30 scd30;
12
13 void setup() {
14     Serial.begin(115200); // Initialisation de la communication série
15     while (!Serial) delay(10);
16
17     Serial.println("Initialisation du capteur SCD30...");
18     if (!scd30.begin()) {
19         Serial.println("Erreur : Impossible de détecter le capteur SCD30 !");
20         while (1) delay(10); // Boucle infinie en cas d'erreur
21     }
22     Serial.println("SCD30 détecté avec succès !");
23
24     Serial.println("Initialisation de LoRaWAN...");
25     lora.init(); // Initialisation de la communication LoRaWAN
26     lora.setId(NULL, const_cast<char *>(DevEUI), const_cast<char *>(AppEUI));
27     lora.setKey(NULL, NULL, const_cast<char *>(AppKey));
28     lora.setDeciveMode(LWOTAA); // Configuration en mode OTAA (Over-the-Air Activation)
29     lora.setDataRate(DR0, EU868); // Définition du débit de données pour la région EU868
```

Code_sae301.ino

```

31     int joinAttempts = 0;
32     const int maxJoinAttempts = 10;
33     Serial.println("Tentative de connexion au réseau LoRaWAN...");
34     while (!lora.setOTAAJoin(JOIN)) { // Tentative de connexion OTAA
35         Serial.println("Échec de la jointure. Nouvelle tentative...");
36         delay(5000);
37         joinAttempts++;
38         if (joinAttempts >= maxJoinAttempts) {
39             Serial.println("Échec de la jointure après 10 tentatives. Redémarrage...");
40             NVIC_SystemReset(); // Redémarrage du microcontrôleur en cas d'échec
41         }
42     }
43     Serial.println("Connexion OTAA réussie !");
44 }
45
46 void loop() {
47     if (scd30.dataReady()) { // Vérifie si des données sont disponibles
48         Serial.println("Données disponibles !");
49         if (!scd30.read()) { // Lecture des données du capteur
50             Serial.println("Erreur lors de la lecture des données du capteur !");
51             return;
52         }

```

Dans notre 2eme Fonction principale loop():

- On pourra lire **les données du capteur** et vérifier si de nouvelles données sont disponibles.
- On pourra également afficher les mesures (CO2, température, humidité) sur le moniteur série.
- **Et donc après affichage**, les données sont converties en un tableau d'octets (payload) compatible avec LoRaWAN avec chaque mesure encodée sur 2 octets.
- **Après on aura la transmission du payload au réseau LoRaWAN avec un message** pour indiquer si l'envoi a réussi ou échoué.
- **Et enfin il y'a la Gestion des boucles. Donc**, si aucune donnée n'est prête, un message d'attente est affiché et une pause de 2 secondes est ajoutée entre chaque itération.

Code_sae301.ino

```

45
46 void loop() {
47     if (scd30.dataReady()) { // Vérifie si des données sont disponibles
48         Serial.println("Données disponibles !");
49         if (!scd30.read()) { // Lecture des données du capteur
50             Serial.println("Erreur lors de la lecture des données du capteur !");
51             return;
52         }
53
54         // Affichage des valeurs mesurées
55         Serial.print("CO2: ");
56         Serial.print(scd30.CO2);
57         Serial.println(" ppm");
58
59         Serial.print("Température: ");
60         Serial.print(scd30.temperature);
61         Serial.println(" °C");
62
63         Serial.print("Humidité: ");
64         Serial.print(scd30.relative_humidity);
65         Serial.println(" %");

```

Code_sae301.ino

```

67 // Création du payload pour LoRaWAN
68 uint8_t payload[6];
69 uint16_t co2 = (uint16_t)scd30.CO2;
70 uint16_t temp = (uint16_t)(scd30.temperature * 100);
71 uint16_t hum = (uint16_t)(scd30.relative_humidity * 100);
72
73 payload[0] = co2 >> 8;
74 payload[1] = co2 & 0xFF;
75 payload[2] = temp >> 8;
76 payload[3] = temp & 0xFF;
77 payload[4] = hum >> 8;
78 payload[5] = hum & 0xFF;
79
80 // Affichage du payload sous forme hexadécimale
81 Serial.print("Payload: ");
82 for (int i = 0; i < sizeof(payload); i++) {
83     Serial.print("0x");
84     Serial.print(payload[i], HEX);
85     Serial.print(" ");
86 }
87 Serial.println();

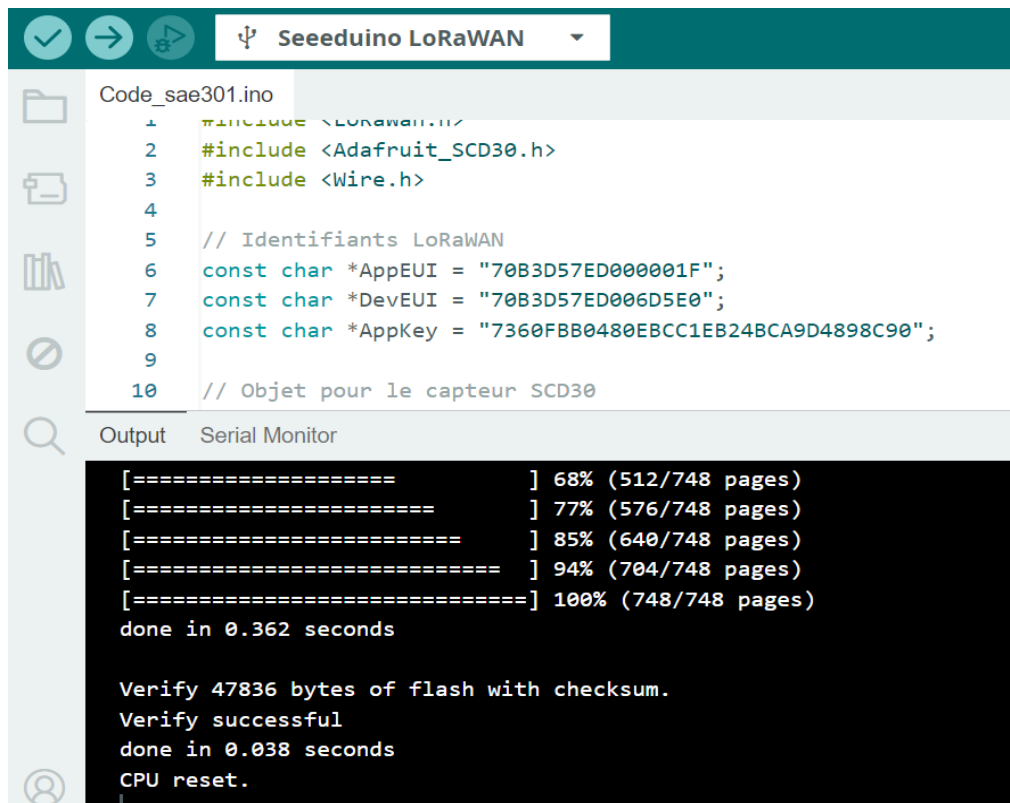
```



```

89      // Envoi des données via LoRaWAN
90      if (lora.transferPacket(payload, sizeof(payload))) {
91          Serial.println("Données envoyées avec succès via LoRaWAN !");
92      } else {
93          Serial.println("Erreur lors de l'envoi des données.");
94      }
95  } else {
96      Serial.println("Aucune donnée disponible pour l'instant.");
97  }
98  delay(2000); // Attente de 2 secondes avant la prochaine lecture
99  }

```



Décode des informations

Au niveau de TTN, le payload est reçu en un tableau de 6 octets (2 octets pour chaque mesure). On aura donc à le decoder au niveau de la section Uplink.

Le décodage des données se fait avec un Custom Javascript Formatter

La fonction décodeur prend deux arguments :

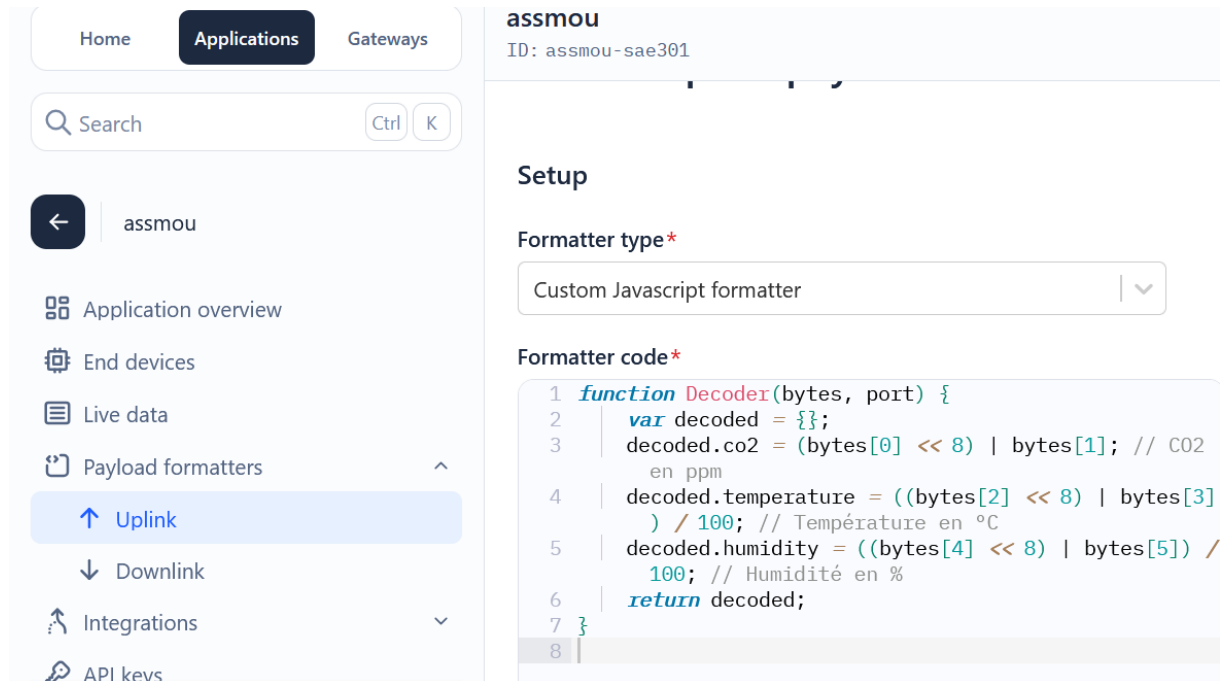
- bytes : le tableau d'octets reçu depuis le capteur.
- port : le numéro de port utilisé pour transmettre les données (utile pour différencier plusieurs types de messages).

Donc, pour décodeur, le programme aura juste à :

- Combiner les octets 0 et 1 pour obtenir la valeur de la concentration en CO2 (en ppm).

- Combiner les octets 2 et 3, puis divise par 100 pour obtenir la température en °C.
- Combiner les octets 4 et 5, puis divise par 100 pour obtenir l'humidité relative en %.

Le décodage final est stocké dans un objet decoded qui est retourné pour les résultats finaux.



assmou
ID: assmou-sae301

Setup

Formatter type*

Custom Javascript formatter

Formatter code*

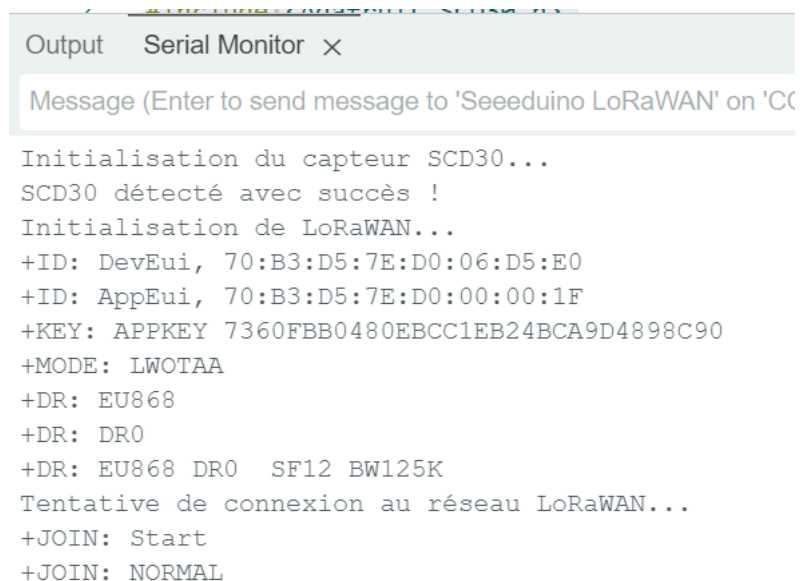
```

1 function Decoder(bytes, port) {
2   var decoded = {};
3   decoded.co2 = (bytes[0] << 8) | bytes[1]; // CO2
    en ppm
4   decoded.temperature = ((bytes[2] << 8) | bytes[3]
    ) / 100; // Température en °C
5   decoded.humidity = ((bytes[4] << 8) | bytes[5]) /
    100; // Humidité en %
6   return decoded;
7 }
8

```

Résultats

Visualisation des données transmises.



Output Serial Monitor x

Message (Enter to send message to 'Seeeduino LoRaWAN' on 'CC

```

Initialisation du capteur SCD30...
SCD30 détecté avec succès !
Initialisation de LoRaWAN...
+ID: DevEui, 70:B3:D5:7E:D0:06:D5:E0
+ID: AppEui, 70:B3:D5:7E:D0:00:00:1F
+KEY: APPKEY 7360FBB0480EBCC1EB24BCA9D4898C90
+MODE: LWOTAA
+DR: EU868
+DR: DR0
+DR: EU868 DR0 SF12 BW125K
Tentative de connexion au réseau LoRaWAN...
+JOIN: Start
+JOIN: NORMAL

```

Output

Serial Monitor ×

Message (Enter to send message to 'Seeeduino LoRaWAN' c

Données disponibles !
CO2: 2086.90 ppm
Température: 18.68 °C
Humidité: 65.12 %
Payload: 0x8 0x26 0x7 0x4B 0x19 0x70
+MSGHEX: Done
+MSGHEX: Start
Données envoyées avec succès via LoRaWAN !
Données disponibles !
CO2: 2083.47 ppm
Température: 18.71 °C
Humidité: 64.98 %
Payload: 0x8 0x23 0x7 0x4E 0x19 0x61
+MSGHEX: Done
+MSGHEX: Start
Données envoyées avec succès via LoRaWAN !

assmou

ID: assmou-sae301

📶 Last activity just now
📶 1 End devices
☆

ENTITY ID

TYPE

DATA PREVIEW

Verbose stream 🔇

Export as JSON

🔍

🗑️

mouass-sae301	Forward uplink data message	DevAddr: 26 0B C0 57 Payload: { co2: 2329, humidity: 63.16, temperature: 18.6 }
mouass-sae301	Forward uplink data message	DevAddr: 26 0B C0 57 Payload: { co2: 2328, humidity: 63.16, temperature: 18.62 }
mouass-sae301	Forward uplink data message	DevAddr: 26 0B C0 57 Payload: { co2: 2328, humidity: 63.18, temperature: 18.62 }
mouass-sae301	Forward uplink data message	DevAddr: 26 0B C0 57 Payload: { co2: 2326, humidity: 63.2, temperature: 18.6 }
mouass-sae301	Forward uplink data message	DevAddr: 26 0B C0 57 Payload: { co2: 2325, humidity: 63.16, temperature: 18.62 }
mouass-sae301	Forward uplink data message	DevAddr: 26 0B C0 57 Payload: { co2: 2326, humidity: 63.05, temperature: 18.59 }
mouass-sae301	Forward uplink data message	DevAddr: 26 0B C0 57 Payload: { co2: 2325, humidity: 63.05, temperature: 18.62 }

Création device, clé, dashboard sur TAGOIO

Pour pour le traitement et l'affichage des données, on utilisera TAGOIO comme solution d'intégration.

Le lien entre TTN et TagoIO s'établit via un **webhook** configuré dans TTN, où l'URL fournie par TagoIO est utilisée pour transmettre automatiquement les données du device. TTN envoie ces données au format JSON vers TagoIO, qui les traite et les associe à un **device** spécifique identifié par une **key authorization**.



All done!

- ✓ Creating device
- ✓ Creating and linking a bucket

Service Authorization				Generate	
Create authorization to allow integration with other services. Learn more .					
Name	Additional Parameter	Created at	Authorization		
Capteur_Key		il y a quelques secondes			

Création des Webhooks

Nous avons configuré des webhooks pour automatiser certaines actions et transmettre les données vers d'autres plateformes.

END DEVICES

Live data

Payload formatters

Uplink

Downlink

Integrations

MQTT

Webhooks

API keys

General settings

Resources

eu1 • v3.33.0.3c8e962fdd

TagoIO

Setup webhook for TagoIO

Integrate with TagoIO

[About TagoIO](#)

Webhook ID*

capteur-weebhook

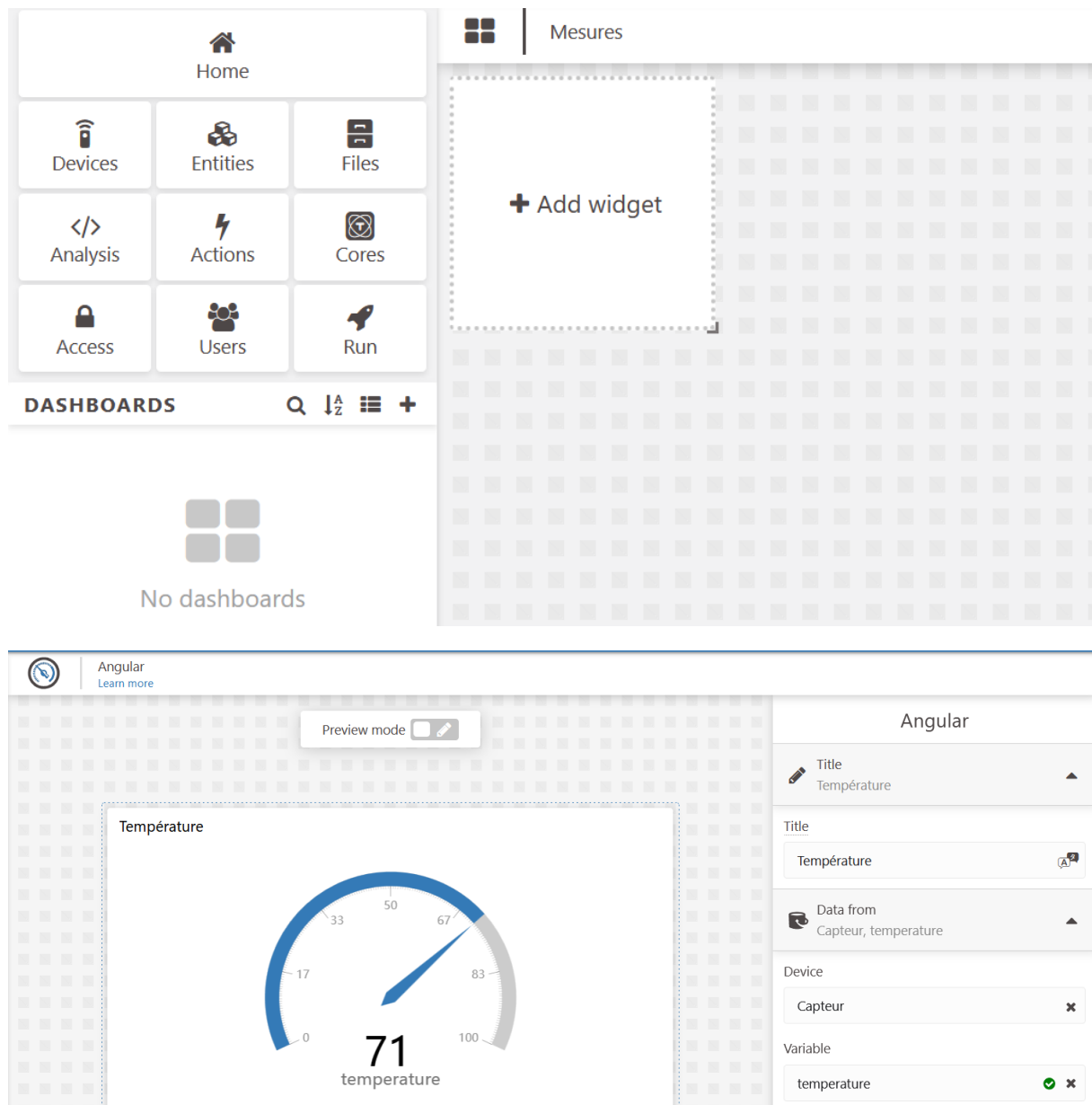
Authorization*

.....

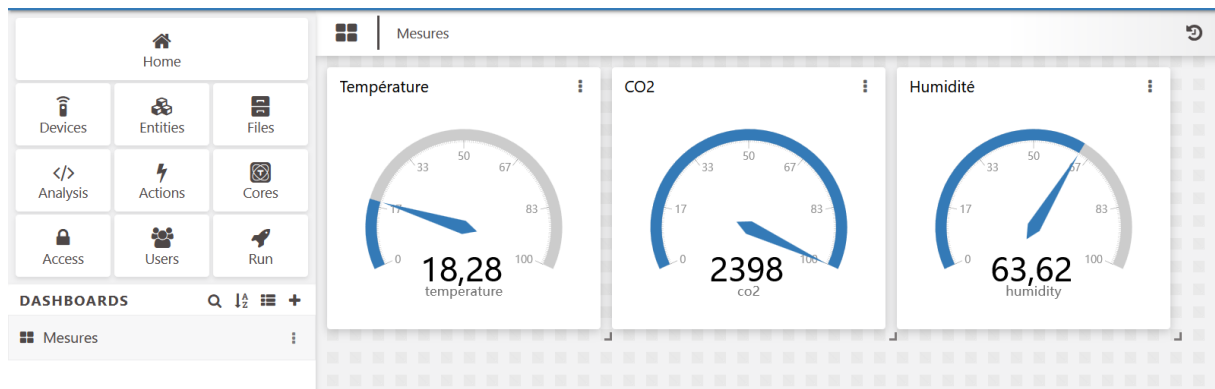
TagoIO Authorization

Create TagoIO webhook

L'interface **dashboard** est un espace personnalisable conçu pour visualiser et analyser les données IoT en temps réel. Elle est composée de **widgets**, des éléments graphiques interactifs comme des graphiques, des jauges ou des tableaux, qui permettent de présenter les données de manière claire.



L'interface **dashboard** est un espace personnalisable conçu pour visualiser et analyser les données IoT en temps réel. Elle est composée de **widgets**, des éléments graphiques interactifs comme des graphiques, des jauges ou des tableaux, qui permettent de présenter les données de manière claire.



Prise d'initiative et améliorations

Configuration automatiques

Une action est définie pour surveiller un capteur spécifique, avec un déclencheur basé sur une condition : lorsque la température est inférieure à 25°C. Lorsqu'elle est activée, une notification par e-mail est envoyée à une adresse prédéfinie avec un message d'alerte personnalisé. Cela permet de recevoir des mises à jour en temps réel et de réagir rapidement à des seuils critiques, assurant ainsi un suivi efficace des données captées.

Trigger	Action
☒ Single device Watch a single device. ☐ Multiple devices Watch all devices with matching tags. Select the device Capteur Trigger If one of the conditions match, the action will be triggered. temperature ☒ is Less than 25 Trigger Unlock (optional) Unlocks are conditions that enable the action to be triggered again.	Name Notification de Température Type of action Send Email Send to assane.ndiaye@etu.univ-st-etienne.fr Title Alerte Température



IUT Roanne's App<0f751@6781200e749eba00099b1c1e.tago.run>



À : ✖ Assane Ndiaye

Dim 26/01/2025 17:29

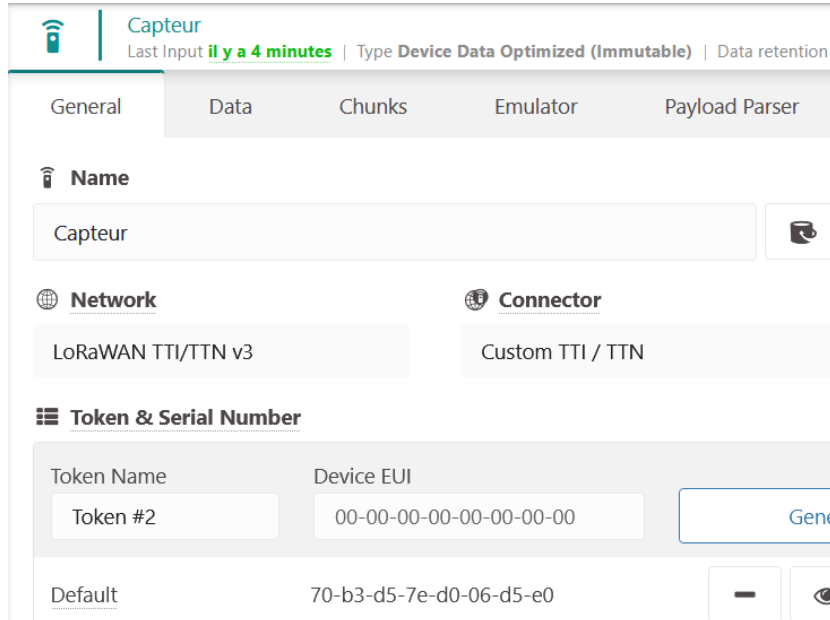
Couvrez-vous, la température à Roanne est actuellement inférieure à 20°C

← Répondre

→ Transférer

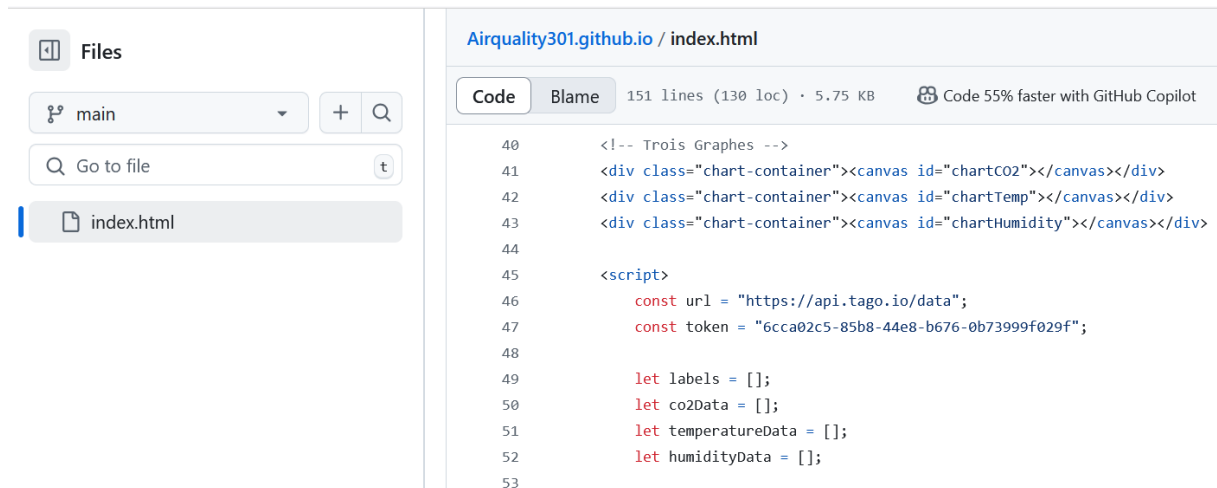
Intégration sur GitHub

L'intégration sur un site hébergé via **GitHub Pages** consiste à créer un fichier HTML contenant du code JavaScript pour récupérer les données du device TagoIO en utilisant l'**API REST** et le **token** associé au device



The screenshot shows the TagoIO configuration page for a device named 'Capteur'. The interface includes tabs for General, Data, Chunks, Emulator, and Payload Parser. Under the General tab, the Name is 'Capteur'. The Network is set to 'LoRaWAN TTI/TTN v3' and the Connector is 'Custom TTI / TTN'. The Token & Serial Number section shows a Token Name of 'Token #2' and a Device EUI of '00-00-00-00-00-00-00-00'. A 'Generate' button is visible next to the EUI field. Below this, a table shows a 'Default' token with EUI '70-b3-d5-7e-d0-06-d5-e0'.

Une fois le fichier HTML ajouté au dépôt GitHub, le site devient accessible via une URL publique, permettant d'afficher dynamiquement les mesures IoT directement sur le site.

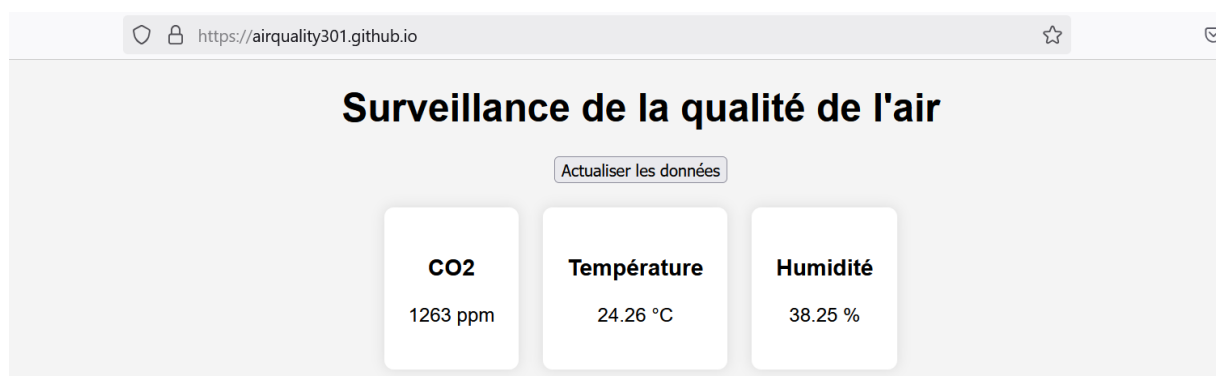


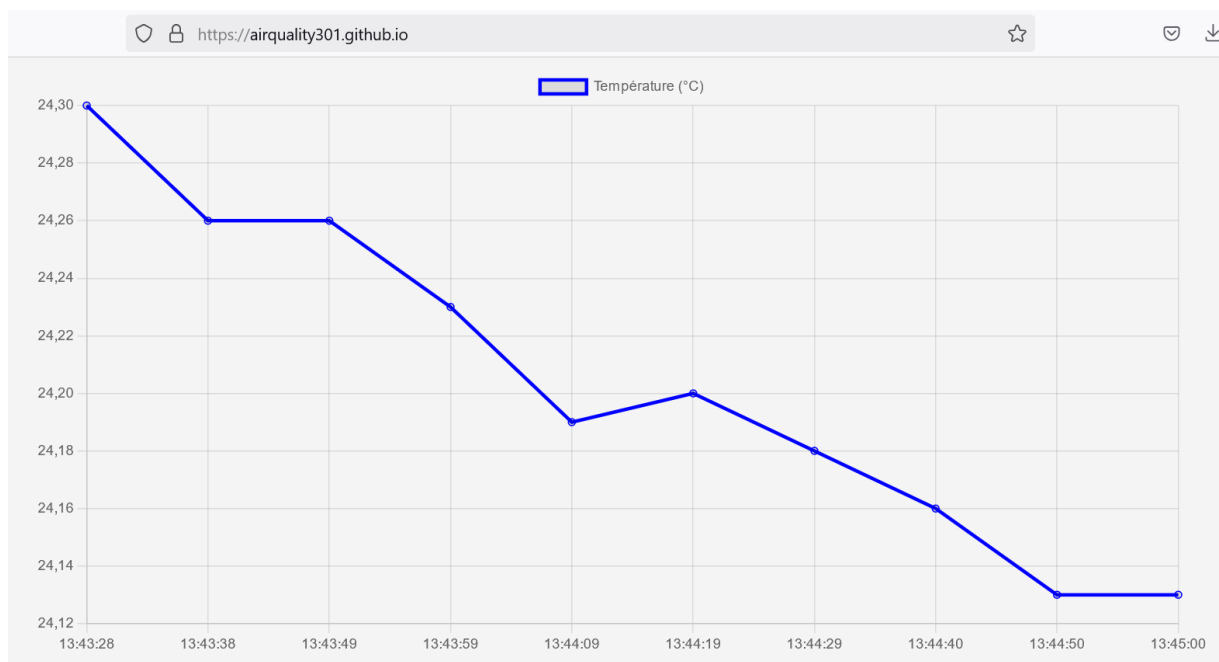
The screenshot shows the GitHub repository interface for 'Airquality301.github.io'. The 'index.html' file is selected, and the code is displayed. The code is a JavaScript file that fetches data from the TagoIO API and displays it in three charts: CO2, Temperature, and Humidity.

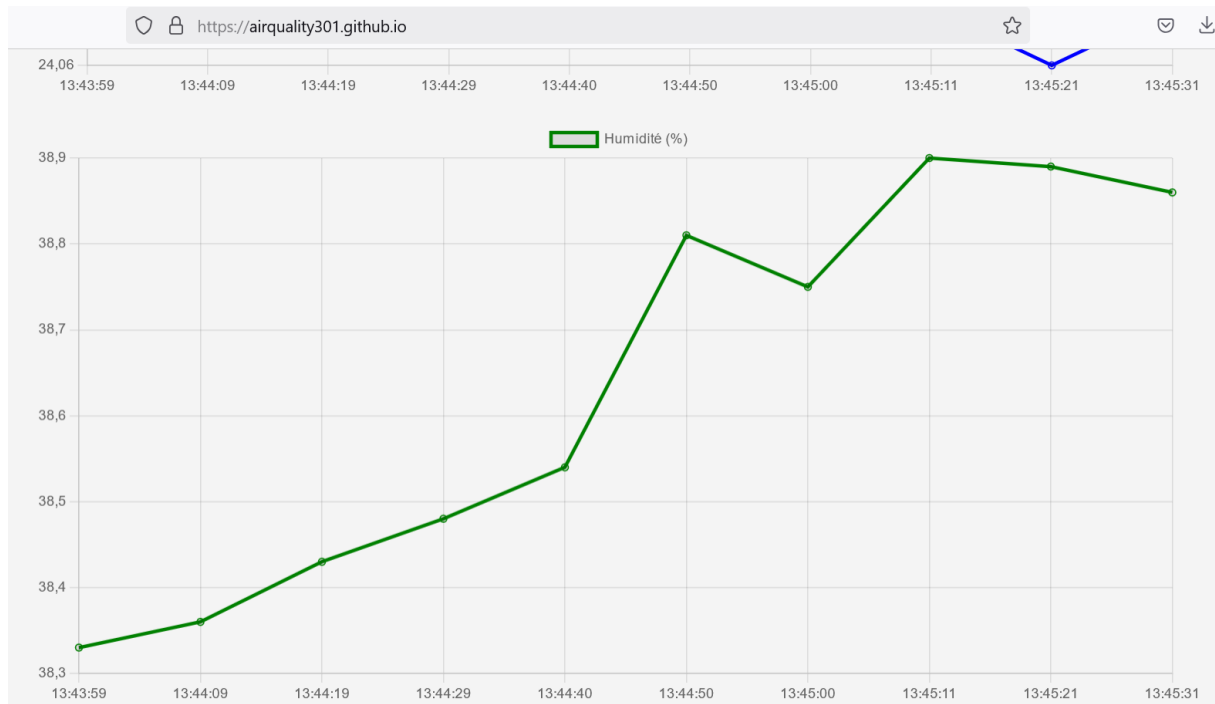
```

40 <!-- Trois Graphes -->
41 <div class="chart-container"><canvas id="chartCO2"></canvas></div>
42 <div class="chart-container"><canvas id="chartTemp"></canvas></div>
43 <div class="chart-container"><canvas id="chartHumidity"></canvas></div>
44
45 <script>
46   const url = "https://api.tago.io/data";
47   const token = "6cca02c5-85b8-44e8-b676-0b73999f029f";
48
49   let labels = [];
50   let co2Data = [];
51   let temperatureData = [];
52   let humidityData = [];
53

```







Conclusion

En conclusion, ce projet a permis de concevoir une solution IoT complète pour mesurer et visualiser la qualité de l'air, en intégrant un capteur SCD30, le réseau LoRaWAN via TTN, et la plateforme TagoIO pour le traitement et l'affichage des données. Le système a démontré son efficacité en assurant une transmission fiable des données et une visualisation intuitive via un site web hébergé sur GitHub Pages. Ce projet constitue une base solide pour des applications IoT futures. En termes de perspectives, il serait intéressant d'intégrer une solution avec **ChirpStack** qui représente une évolution intéressante. Cela permettrait de déployer une infrastructure LoRaWAN privée, offrant un contrôle total sur le traitement des données, une meilleure personnalisation, et l'intégration avec des outils avancés comme Grafana pour des visualisations plus poussées.